

CSED101 Programming and Problem Solving

Lecture note for CSED101 Finalterm

Hyunseong Kong, POSTECH 24

Last updated: March 6, 2025

Contents

0. AI Project	2
0.1 Intruduction	2
0.2 Machine Learning Type	2
0.3 Examples	2
1. Data Structure - 데이터구조	3
1.1 Iterable Object	3
1.2 Further Data Structures	3
2. Object Oriented Programming - 객체지향	4
2.1 객체지향 프로그래밍	4
2.2 추상 데이터 타입	4
2.3 클래스	5
2.4 객체지향의 특징들	6
2.5 객체지향 설계	7
2.6 클래스 다이어그램	9
3. Sort and Search - 정렬과 검색	10
3.1 Sort	10
3.2 Search	11
4. Recursion - 재귀	11
4.1 Recursion	11
5. GUI Programming - 그래픽 프로그래밍	12
5.1 TUI vs GUI	12
5.2 Tkinter	12

6. Numpy and Pandas	14
6.1 Numpy	14
6.2 Pandas	16

※ 0. AI Project

0.1 Intruduction

- **인공지능***Artifitial Intelligence*: 컴퓨터가 사람처럼 생각하고 판단하게 만드는 기술
- **머신러닝***Machine Learning*: 인공지능의 한 분야. 인간의 학습 능력과 같은 기능을 컴퓨터에 부여하기 위한 기술
- **딥러닝***Deep Learning*: 인공신경망 기반으로 한 머신러닝의 방법 중 하나. 빅데이터 기반으로 스스로 학습하여 판단하는 기술

파이썬으로 인공지능을 코딩하는 것은 관련한 다양한 라이브러리와 같은 여러 요인들로 인해 간편하다는 장점이 있다.

0.2 Machine Learning Type

- **지도 학습***Supervised Learning*: 문제와 정답을 알려주고 공부. (키워드: 예측, 분류)
- **비지도 학습***Unsupervised Learning*: 답을 가르쳐주지 않고 공부. (키워드: 연관 규칙, 군집)
- **강화 학습***Reinforcement Learning*: 보상을 통해 상을 최대화, 벌을 최소화하는 방향으로 학습. (키워드: 보상)

0.3 Examples

- **Computer Vision**: 컴퓨터가 이미지나 비디오 해석
 - **Object Detection**: 이미지에서 의미 있는 객체의 종류와 위치(bounding box) 찾을
 - **Object Tracking**: 비디오에서 움직이는 객체를 식별하고 추적
 - **Face Recognition**: 이미지, 비디오에서 얼굴을 인식하고 식별
 - * 보로노이 알고리즘: 얼굴에 점을 찍어서 얼굴 인식함.
 - **Image Generation**: 텍스트나 다른 이미지 기반으로 새로운 이미지 생성

- NLP: 컴퓨터가 인간의 언어를 이해하고 생성함
 - **Machine Translation:** 번역
 - **Chatbot and Conversational System:** 챗봇 및 대화. chatbot은 데이터가 부족하여 영화 대본을 기반으로 만들.
 - **Sentiment Analysis:** 감정 분석
 - **Text Summarization:** 텍스트 요약

AI 활용 예시: 헬로미러, 얼굴 인식 인공지능 스피커, 자율주행 및 장애물 회피(POSLA, RC Car, 자동 쇼핑카트), 군집 주행 드론, 주가 예측, 시험 부정 방지, 메타버스 토론

※ 1. Data Structure

1.1 Iterable Object

리스트 컴프리헨션List Comprehension을 통해 여러 리스트를 쉽게 만들 수 있다.

- **이터러블^{Iterable}:** 순회 가능한 객체로 list, tuple, string 등이 해당된다.
- **아이템^{Item}:** iterable에서 현재 항목을 나타낸다.
- **표현식^{Expression}:** 현재의 item에 대한 연산이나 처리 결과를 나타낸다.
- **필터링^{Filtering}:** 조건식을 넣어 값에 대한 필터링이 가능하다.

즉, 아래와 같이 쓸 수 있다.

[표현식 **for** 아이템 **in** 이터러블 **if** 조건]

또한, 조건을 만족하지 않았을 때의 별도의 값도 아래와 같이 입력하여 저장이 가능하다.

[표현식 **if** 조건 **else** 값 **for** 아이템 **in** 이터러블]

1.2 Further Data Structures

collections은 파이썬 내장 모듈로, 다양한 데이터 구조들을 지원한다. 아래는 그 중 여러 예시이다.

- **스택^{Stack}**: LIFO 구조이다.
- **큐^{Queue}**: FIFO 구조이다.
- **디큐^{Deque}**: 양쪽 끝에서 삽입과 삭제가 모두 가능하다. 스택과 큐를 모두 지원하는 것이다.
- **순서딕셔너리^{OrderedDict}**: 입력된 순서대로 key-value를 저장한다. key 혹은 value로 데이터 정렬이 가능하다.
- **계수기^{Counter}**: 데이터 값의 개수를 dictionary 형태로 반환한다. 리스트 안에 중복되는 데이터가 있으면 {데이터}:[개수],... 형태의 리스트로 반환하는 함수이다.

※ 2. Object Oriented Programming

2.1 객체지향 프로그래밍

- **절차지향 프로그래밍^{Procedural Programming}**: 프로그램을 구성하는 데이터^{Data}와 절차^{procedure}를 분리하여 접근한다. 명령어의 순서와 함수 호출에 따라 프로그램이 작동되며, 함수는 입력된 인수 가지고 작동한 뒤 결과를 반환한다. 단순하기 때문에 구조화하기가 쉽다.
- **객체지향 프로그래밍^{Object Oriented Programming}**: 프로그램을 객체(데이터와 함수를 하나로 묶은 것)의 모임으로 접근한다. 객체는 각자의 특성이 존재하며, 모듈화가 쉽다.

즉, 절차지향 프로그래밍은 함수에 보다 중점을 둔 코딩 방법이고 객체지향 프로그래밍은 객체에 중점을 둔 코딩 방법이다.

여기서 중요한 점은 객체지향이 기능과 데이터를 묶은 하나의 단일 객체를 만드는데 주 목적을 두고 있다는 것이다. 가령, 자동차 추돌 사고가 있었을 때, 우리는 두 개의 자동차라고 하는 객체의 상호작용으로서 이를 해석한다. 객체지향의 아이디어는 여기에서 시작한다. 즉, 객체지향은 사람이 사물을 바라보는 관점으로 프로그램을 설계하고자 하는 관점인 것이다.

2.2 추상 데이터 타입

추상 데이터 타입^{Abstract Data Type, ADT}은 데이터와 데이터에 적용할 수 있는 연산의 모임으로 구성된 수학적 모델이다. 여기서 하고자 하는 것은, 모델의 논리적인 부분을 정의하고 데이터와 연산의 관계를 "추상화"하는 것이다.

이 단원에서 말하고자 하는 **추상화^{Abstraction}**는 어떠한 대상에서 꼭 필요한 것만 남기고

나머지를 무시하는 방식으로 진행한다. 이 과정에서 객체의 **속성**^{Property}과 **기능**^{Method}을 명확히 파악하고 구분해야 한다. 예를 들어, 자동차라고 하는 객체를 만들어 충돌 시뮬레이션을 하는 상황에서 자동차의 연비는 전혀 중요하지 않다. 즉, 이 상황에서 정의된 자동차 객체는 연비에 관한 데이터를 포함하지 않게 된다.

ADT는 데이터와 연산을 분리하고 내부 구현을 감춘다. 이를 **캡슐화**^{Encapsulation}이라고 부른다. 내부 구현을 감춘다는 것은 객체를 사용하는 사람이 내부 구현 코드가 실제로 어떻게 되어있는지 알 필요가 없다는 의미이다. 예전에 배웠던 set의 경우, 실제 구현은 최적화를 포함하여 다양한 알고리즘을 통해 구현되어 있다. 그러나, 우리가 set을 사용하는데 이러한 최적화를 전부 알 필요가 없다. 그저 set을 만들고, 교집합, 차집합과 같은 연산들만 알면 된다. 이것이 내부 구현을 감춘다는 것의 의미이다.

ADT는 수학적 모델에 불과하기에 객체지향에서는 만들어진 ADT를 실제 코드로 구현하여 사용하기 위해 **클래스**^{Class}라고 하는 키워드를 지원한다.

ADT를 사용하였을 때의 장점을 보다 구체적으로 정리하자면 아래와 같다.

- **모듈화와 추상화**: 코드 가독성과 유지보수 및 재사용성 증가된다.
- **재사용성**: ADT 통해서 만든 데이터 구조와 연산은 여러 프로그램에서 재사용이 가능하며, 데이터 구조 변경하지 않고 연산을 수정하거나 대체할 수 있다.
- **알고리즘과 데이터 구조의 독립성**: 서로 독립적인 개발이 가능하며, 코드의 유연성 증가되고, 새로운 알고리즘을 동일한 데이터 구조에 적용할 수 있다.
- **인터페이스 명확성**: 데이터와 연산의 인터페이스 명확하게 정의할 수 있기 때문에 프로그램의 구조 이해가 쉽다.
- **인터페이스**^{Interface}: 두 대상이 정보를 주고 받는 경계
- **public**: 파이썬의 기본 정보 접근 제한자이며, 클래스 밖에서 접근을 허용한다.
- **private**: 파이썬에서는 이름 앞에 "_"를 붙여서 클래스 밖에서의 접근을 막을 수 있다.

2.3 클래스

파이썬의 **클래스**^{Class}는 데이터, 함수를 묶어서 선언할 수 있도록 만들어진 키워드로, 프로그래머가 커스텀 자료형을 만들 수 있도록 한다. 파이썬은 모든 대상을 **객체**^{Object}라고 하는 개념을 사용하여 저장한다. 기본 데이터 타입도 전부 객체이며, 각 객체는 Identity, Type, Value를 갖는다. 클래스에서 데이터를 선언하는 위치를 **필드**^{Field}라고 한다. 클래스에서 기능은 함수로 구현되며, 이렇게 클래스 내부에서 구현된 함수를 **메서드**^{Method}라고 부른다. 모든 메서드는 내부 인스턴스의 요소들을 불러올 수 있게 self 매개변수를 갖는다.

클래스 자체는 객체를 만들 수 있는 하나의 틀이며, 이에 클래스로 만든 실제 객체를 클래스 자체와 구분하여 **인스턴스**^{Instance}라고 부른다. 각 인스턴스는 별개의 필드를 가지기 때문에 각 인스턴스가 전부 다른 필드 값을 가지는 것이 가능하다.

클래스는 다음과 같은 여러 특수 메서드가 있다.

- **__init__**: **생성자**^{Constructor}이다. 생성자는 인스턴스 생성 시에 무조건적으로 호출되는 특수 메서드이다. 기본 constructor는 매개변수가 self만 있다.
- **__del__**: **소멸자**^{Destructor}이다. 소멸자는 인스턴스 소멸 시에 무조건적으로 호출되는 특수 메서드이다.
- **__str__**: print()의 매개변수로 인스턴스가 들어간 경우, 이 메서드의 반환 값이 들어가 print 된다.
- **__len__**: len()의 매개변수로 인스턴스가 들어간 경우, 이 메서드의 반환 값이 len 함수의 반환값이 된다.

각각의 인스턴스마다 할당된 변수를 **인스턴스 변수**^{Instance Variable}라고 한다. 인스턴스 변수를 사용하기 위해서는 인스턴스.변수를 이용하여 값을 가져올 수 있다.

같은 클래스로 만들어진 모든 인스턴스가 공유하는 변수를 **클래스 변수**^{Class Variable}라고 부른다. 클래스 변수는 클래스.변수로 값을 가져올 수 있다.

클래스 변수는 필드에, 인스턴스 변수는 함수 __init__안에 self를 붙여서 선언하는 것이 일반적이지만, 둘 다 필드에 선언한 후 인스턴스.변수 = 10과 같이 값을 할당하면 클래스 변수가 **가려지고**^{Shadowing} 인스턴스 변수로 바뀌게 된다.

2.4 객체지향의 특징들

- **캡슐화**^{Encapsulation}: 객체의 **상태**^{State}, **행위**^{Behavior}를 하나로 묶고, 외부에서의 접근을 제한하여 객체를 보호하는 과정. 캡슐화로 객체의 내부 구조를 감추고 외부에 제공하는 인터페이스만 노출
- **정보 은닉**^{Information Hiding}: 데이터 보호 위해 다른 객체 접근 제한하는 접근 제한 수식자 기능 제공
- **상속**^{Inheritance}: 기존 클래스 필드와 메소드를 물려받아 그것을 기반으로 새로운 클래스 만드는 과정. 기존 클래스 속성과 기능을 재사용하고 새로운 기능 추가하여 코드 간결하게 작성 가능. 이때, 상위 클래스를 **슈퍼 클래스**^{Super Class}, **부모 클래스**^{Parent Class}라고 하고, 하위 클래스를 **부클래스**^{Subclass}, **자식 클래스**^{Child Class}라고 한다.
- **메서드 오버라이딩**^{Method Overriding}: 부모 클래스의 메서드를 자식 클래스에서 재정의하는 것.

- **연산자 오버로딩** *Operator Overloading*: 필요한 연산자를 내장 타입과 형태와 동작이 유사하도록 재정의하는 것.
- **다형성** *Polymorphism*: 같은 이름의 메서드가 서로 다른 객체에서 다르게 동작할 수 있는 특성. 다형성을 통해 객체간의 상호작용을 쉽게 구현한다.

부모 클래스와 자식 클래스의 같은 메서드 호출에 대해 다르게 반응하도록 하는 것은 다형성의 예시이다. 즉, 이를 가능케하는 메서드 오버라이딩, 연산자 오버로딩이 다형성을 지원하는 방법 중 하나이다.

- 다형성 장점: 적은 코딩으로 다양한 객체에서 유사한 작업 수행 가능하며 코드의 가독성을 높여준다.
- python은 자료형을 명시하지 않아도 되기 때문에 더 쉽게 다형성을 구현해낼 수 있다. 실시간으로 객체의 자료형이 결정되므로 단 하나의 메서드에 의해 여러 객체를 처리할 수 있음.

아래는 연산자 오버로딩에 대응되는 함수들이다. 각각은 `__`를 양쪽에 붙여서 특수 함수 꼴로 만든 다음에 이용해야 한다.

- 산술: `add, sub, mul, truediv, floordiv, mod, divmod`(몫, 나머지를 tuple로 반환), `pow, neg, lshift`
- 관계: `lt, le, eq, ne, gt, ge`
- 논리: `and, or, xor, invert(~)`
- 인덱스 접근: `getitem, setitem`

피연산자 순서가 바뀌는 경우, 이름에 "r"을 붙인 메서드를 정의한다. 즉, `object + 100`은 `add`에서 처리하며, `100 + object`은 `radd`에서 처리한다.

`=`가 붙어있는 연산자는 이름에 "i"를 붙인 메서드를 정의한다. 즉, `a += b`는 `iadd`에서 처리한다. 이때, 연산 수행후 `self`나 인스턴스 등을 반환해야 한다. 이는 `v1 += v2`가 `v1 = v1 + v2`로 쪼개져 동작하기 때문이다. 즉, `iadd`는 값을 더한 뒤에, 반환한 값이 `v1`에 들어가는 형식이다.

2.5 객체지향 설계

객체지향 설계 *OOP Design*는 요구 사항을 분석하고 이를 효율적으로 구현할 수 있는 클래스를 설계하는 과정이다.

1. 요구 사항 분석

- 문제 이해하기: 시스템이 해결해야 할 문제를 이해
- 기능적 요구 사항 정의: 시스템이 수행해야 할 기능을 명확히 도출
- 비기능적 요구 사항 정의: 성능, 보안, 사용성 등의 비기능적 요소를 고려

2. 개념 모델링

- 도메인 개념 식별: 시스템과 관련된 주요 개념들을 식별
- 속성 정의: 각 개념의 속성을 정의
- 관계 파악: 개념들 사이의 관계를 파악

3. 클래스 도출

- 클래스 후보 선정: 개념 모델링에서 식별된 개념들을 바탕으로 클래스 후보를 선정
- 클래스 정의: 각 클래스의 속성(개념 모델링 속성)과 메서드(시스템 기능적 요구사항)를 정의
- 상속 및 연관 관계 설정: 클래스 간의 상속, 연관, 집합, 구성 등의 관계 정의

4. 반복 및 정제

- 반복적 검토 및 수정: 설계는 반복적인 과정이며, 새로운 요구사항이 추가되거나 변경될 때마다 클래스를 재검토하고 필요에 따라 수정
- 정제: 중복을 제거하고, 불필요한 클래스를 줄이며, 클래스의 책임을 명확하게 작성

5. 문서화 및 검증

- UML 다이어그램 작성: 클래스 다이어그램을 통해 클래스와 그 관계를 시각적으로 표현
- 검증: 설계가 모든 요구사항을 충족하는지 검증

OOP 설계에서의 주의사항은 다음과 같다.

- **실세계 모델링:** OOP 설계는 실세계 문제를 모델링하는 것이므로 실세계의 개념과 프로세스를 잘 이해하는 것이 중요하다.
- **유연성 유지:** 요구 사항 변경에 유연하게 대응할 수 있는 설계를 고려해야 한다.
- **과도한 세분화 방지:** 너무 많은 클래스로 세분화하면 복잡성이 증가되므로 주의한다.

2.6 클래스 다이어그램

클래스 관계는 클래스 관계도를 그릴 때 아주 중요한 개념이다. 각 관계를 이해하는 것은 코드를 보다 명확하게 인식하는 데에도 영향을 준다. 배우는 클래스 관계는 총 4가지가 있으며, 각각은 다음과 같다.

1. **상속**^{Inheritance} (is-a relation): 한 클래스가 다른 클래스를 가져온 뒤, 일부 구현을 수정하거나 없는 기능을 넣어 변화를 주는 것이다. 무엇인가가 '종속'된다는 개념과는 무관하다.
 - UML 관계도: 실선 & 화살표
 - 예시: 포유류와 고래 클래스. 고래 클래스를 만드려면 포유류 클래스를 상속해야 한다. 고래는 포유류이지만, 포유류가 고래는 아니다.
2. **연관**^{Association} (use-a relation): 서로가 서로에게 소유권이 없기 때문에 종속 관계가 아니다. 서로를 알고 있지만 서로의 생명 주기에는 영향이 없다. 한 인스턴스가 다른 인스턴스를 일방적으로 쓰는 것이 보편적인 구현 방식이기 때문에 has-a relation으로도 표현된다.
 - UML 관계도: 점선
 - 예시: 선생님과 학생 클래스. 서로 관련은 있지만 서로에게 속해있지는 않다. 서로가 서로의 메서드나 데이터에 영향을 주진 않는다.
3. **집약**^{Aggregation} (has-a relation): 연관에 whole-part 관계를 추가했다. 이 관계는 하나의 메인 클래스가 다른 하위 클래스들을 포함하는 관계이다. 메인 클래스가 하위 클래스에 대해 약한 소유권을 가지며, 하위 클래스가 메인 클래스에 종속되지는 않기에 서로의 생명 주기에 영향은 주지 않는다.
 - UML 관계도: 실선 & 빈 마름모
 - 예시: 회사 부서와 회사원 클래스. 한 부서 안에 회사원들을 묶을 수는 있지만 완전히 종속된 것은 아니다. 부서는 언제든지 옮길 수 있고, 부서가 없어져도 구성원들이 없어지지는 않는다.
4. **구성**^{Composition} (owns-a relation): 집약과 비슷하지만 메인 클래스가 강한 소유권을 갖는다. 즉, 클래스가 종속되기에 메인 클래스가 하위 클래스의 생명 주기에 영향을 준다. 집약이기도 하기 때문에 has-a relation이기도 하다.
 - UML 관계도: 실선 & 채워진 마름모
 - 예시: 집과 방 클래스. 집 안에 여러 방이 속해있고, 집이 없어지면 마찬가지로 집의 하위 요소였던 방도 동시에 없어진다.

위의 4가지 관계 중 집약과 구성을 합쳐서 집합 관계라고 부른다.

연관과 집약은 특성상 코드 구현에서 큰 차이가 없는 경우가 있다. 그러나, 둘은 의미상의 차이점이 분명히 존재한다. 의미상의 차이점은 코드를 더 명확히 이해하기 위함이기 때문에 굉장히 중요한 개념이다.

연관은 보통 인스턴스를 메서드 매개변수로 넘겨서 한 번씩 쓰는 형태이지만, 집약은 아예 클래스 안에 리스트를 만들어놓고 거기에 주변 클래스를 다 넣어둔 뒤에, 필요할 때마다 꺼내서 쓰는 형태이다.

※ 3. Sort and Search

3.1 Sort

정렬 알고리즘 *Sort Algorithm* 은 입력된 데이터를 특정한 기준에 따라 오름차순 혹은 내림차순으로 값을 정렬하는 알고리즘이다.

- **버블 정렬** *Bubble Sort*: 인접한 두 데이터를 비교하여 정렬하는 방법. 처음부터 끝까지 인접한 원소 비교하여 큰 값을 오른쪽으로 이동시킴. 이렇게 이동시킨 후, 다시 정렬 작업을 반복. 느리지만 간편함.
- **선택 정렬** *Selection Sort*: 배열에서 최솟값 찾아서 맨 앞의 요소와 교체하는 방법.
- **삽입 정렬** *Insertion Sort*: 배열의 두 번째 원소에서 시작하여 앞의 요소들과 비교하여 삽입할 위치를 결정한 다음에 삽입. selection 보다는 빠르지만 데이터 크기가 크면 효율성 떨어짐.
- **퀵 정렬** *Quick Sort*: 분할 정복 알고리즘. pivot 기준으로 큰 값과 작은 값을 나누어 각각에 대해 재귀적으로 정렬을 수행. 대부분의 경우에서 가장 빠른 알고리즘.
- **병합 정렬** *Merge Sort*: 분할 정복 알고리즘. 주어진 배열을 두 부분으로 나눈 뒤, 각각을 정렬한 후에 다시 합치는 방식. 퀵 정렬과 마찬가지로 대부분의 경우에 가장 빠름.
- **팀 정렬** *Tim Sort*: python의 정렬 알고리즘. 작은 부분 배열은 삽입 정렬로 처리하고 정렬된 부분 배열을 합치는데는 병합 정렬을 이용한다. 이미 정렬된 배열이나 거의 정렬된 배열을 매우 빠르게 처리한다.

파이썬은 `sort`, `sorted` 두 함수를 지원한다. `sort`는 리스트 함수이며, 원래 리스트를 변경하고 `None`을 반환한다. 반면, `sorted` 리스트를 받아서 정렬한 결과를 반환하며, 원래 리스트는 변경되지 않는다.

두 함수 모두 `key`라고 하는 매개변수를 지원하며 `key`에 람다 함수를 넣어서 오름차순을 내림차순으로 바꾸거나, 원하는 방식의 순서를 갖도록 정렬을 할 수 있다.

3.2 Search

검색 알고리즘 *Search Algorithm* 은 데이터 리스트에서 원하는 값을 찾는 알고리즘이다.

- **순차 검색** *Sequential Search*: 리스트를 앞에서부터 순서대로 하나씩 비교하여 원하는 값을 찾는다. 정렬되지 않은 배열에도 적용 가능하지만 비효율적이다.
- **이진 검색** *Binary Search*: 정렬된 데이터에서 리스트의 중간 값을 검사하여 찾는 값이 더 크면 오른쪽 절반에서 다시 검색, 작으면 왼쪽 절반에서 다시 검색한다.

※ 4. Recursion

4.1 Recursion

재귀 *Recursion* 은 함수가 자기 자신을 다시 호출하여 문제를 해결하는 프로그래밍의 기법이다.

반복문과 같이 반복적으로 작업을 수행하나 동일한 작업 수행하는 코드를 보다 간결하게 작성할 수 있다. 일부 문제를 해결하기 위해 더 작은 문제로 분할한 뒤에, 이들을 해결하여 전체 문제를 해결하는 과정을 거친다.

함수가 자신을 호출하면서 호출 횟수를 제한하는 조건문이 필수적이며 이러한 조건문이 없을 경우, 무한한 재귀 호출이 발생할 가능성이 있기에 주의해야 한다.

- 장점: 일부 문제에서 반복문에 비해 코드가 간결해진다. 그래프 탐색과 같은 여러 문제들의 해결에 용이하다.
- 단점: 자기 자신 호출하는 과정에서 스택 메모리 **Stack Memory**를 사용하기에 메모리 사용량이 증가하며 반복문보다 함수 호출과 반환에 대한 오버헤드(함수 호출시에 발생하는 추가적인 작업량)가 크기 때문에 실행 속도가 저하된다.

하노이 탑 문제는 3개의 기둥에 크기가 다른 원판들이 여러 개 있을 때, 한번에 하나의 원판을 옮겨서 첫 번째 기둥에 있는 모든 원판을 세 번째 기둥으로 옮기는데 걸리는 최소 반복 횟수를 구하는 문제이다. 이때, 작은 원판 위에 큰 원판을 옮길 수 없으며, 원판을 옮기는 전체 과정을 출력할 수 있어야 한다.

- 재귀적 해법: 초기 상태로 1번 기둥에 n 개의 크기가 다른 원판이 놓여져 있다고 하자. 상위 $n-1$ 개 원판을 2번째 기둥으로 옮긴 다음에, n 번째 원판을 3번째 기둥으로 옮기고, 상위 $n-1$ 개 원판을 3번째로 옮기면 된다.

※ 5. GUI Programming

5.1 TUI vs GUI

텍스트 유저 인터페이스 *Text User Interface, TUI*, **그래픽 유저 인터페이스** *Graphic User Interface, GUI*는 사용자-컴퓨터 간의 상호작용 위한 주요한 인터페이스 유형이다. TUI는 텍스트 기반이다. 콘솔이나 명령 줄로 알려져 있음. 사용자는 명령어를 입력하고 텍스트 기반 정보를 받아들이며 키보드 이용하여 상호작용한다. MS DOS, linux shell, vim 등이 그 예시이다.

- **장점:** 자원 소모가 적고 경량. 텍스트 기반이라 console에서도 사용 가능. 간단하고 빠르며, 스크립트와의 통합이 용이함.
- **단점:** 사용자가 명령어나 옵션 기억하고 입력해야 함. 그래픽 요소 제한적이기에 복잡한 시각적 표현 어려움

GUI는 그래픽 요소를 사용하여 사용자와 상호작용한다. 사용자는 윈도우, 버튼, 메뉴, 입력 필드와 같은 요소를 마우스로 조작한다.

- **장점:** 시각적인 요소로 직관적인 환경을 제공하며, 복잡한 작업을 그래픽으로 간단하게 수행함. 다양한 입력 방식 지원하여 사용자 편의성 높임.
- **단점:** 자원 소모 크고, 구현과 디자인에 시간이 많이 걸림. 다양한 플랫폼에서 동일한 모습을 유지하기가 힘들.

5.2 Tkinter

파이썬은 GUI 프로그램을 제작하는데 쓸 수 있는 tkinter, pyqt, pyside, kivy, wxpython 등의 다양한 도구 지원한다. 이중 tkinter는 파이썬의 표준 라이브러리로 문법이 상대적으로 단순하며 별도의 설치가 필요 없다는 장점을 갖는다. 또한, tkinter는 다음의 특징을 갖는다.

- **크로스 플랫폼 지원:** 파이썬은 크로스 플랫폼 개발을 지원하므로 동일한 코드를 사용하여 여러 플랫폼에서 GUI 애플리케이션을 실행할 수 있다.
- **확장성:** 파이썬은 다른 언어와의 통합이 용이하며, C/C++로 작성된 라이브러리의 결합을 통해 추가적인 기능을 제공할 수 있다. PyQt, wxPython은 Qt, wxWidgets 라는 C++ 기반 프레임워크 이용할 수 있다.
- **풍부한 기능과 위젯:** 다양한 기능과 위젯을 제공하여 사용자 인터페이스를 다양하게 구성할 수 있음. 버튼, 레이블, 텍스트 상자, 체크박스, 라디오 버튼, 드롭다운

메뉴 등의 위젯 활용할 수 있음. 그래픽 요소도 처리할 수 있기 때문에, 도표, 차트, 이미지 표시와 같은 고급 기능을 개발하는데 유용함.

- **데이터 시각화:** matplotlib, seaborn, plotly 등을 이용하여 데이터 시각적으로 표현하고 그래프, 플롯, 차트 등을 생성할 수 있음. 데이터 시각화는 데이터 분석 및 보고에 필수적인 요소로 사용이 가능함.

위젯^{Widget}은 사용자 인터페이스를 구성하는 요소를 통칭하는 말이다. 위젯은 다양한 상호작용 가능한 요소를 포함한다. 즉, 위젯은 사용자와 애플리케이션 간의 상호작용을 담당하며 동시에 애플리케이션 기능 제어하고 정보를 표시하는 역할을 한다.

- **label:** 사용자에게 텍스트나 이미지 표현
- **button:** 사용자가 클릭할 수 있는 버튼
- **entry:** 사용자로부터 텍스트를 입력받을 수 있는 상자
- **checkboxbutton:** 사용자가 체크할 수 있는 옵션을 표시. 여러 개의 체크 박스를 그룹으로 묶어 다중 선택 가능함
- **radiobutton:** 여러 개의 옵션 중 하나를 선택할 수 있는 버튼. 여러 개의 라디오 버튼을 그룹으로 묶어 단일 선택 가능함
- **listbox:** 여러 항목을 리스트 형태로 표시하고 사용자가 선택할 수 있게 하며, 단일 선택과 다중 선택이 가능함
- **combobox:** 텍스트 상자와 드롭다운 목록을 결합한 widget. 사용자가 선택할 수 있는 목록을 제공하며, 자유롭게 텍스트 입력 가능함
- **text:** 여러 줄의 텍스트를 입력하거나 표시하는데 사용. 대용량 텍스트 편집 가능하며 스크롤바 포함 가능함.

다음은 Tkinter를 사용해 GUI 프로그램을 작성할 때 공통적으로 사용하는 명령어들이다. 즉, 최소한 아래의 명령어를 작성해야 Tkinter를 사용해 프로그래밍을 시작할 준비가 된 것이다.

- **window=tk.Tk():** Tk class로 주 창을 생성. 루트 윈도우를 초기화.
- **window.title():** 창 제목 설정
- **window.mainloop():** tkinter 애플리케이션의 이벤트 루프를 실행한다. 이벤트 처리하고 창을 업데이트하는데 사용된다.

위젯을 만들고 프로그램의 흐름을 제어하는 데에는 아래의 명령어를 주로 사용한다.

- 위젯 = tk.Entry(): 입력 창 설정. input_entry.get()으로 입력된 값 받아옴.
- 위젯.pack(): 화면에 배치. 자동으로 배치됨.
- 위젯.config(): 속성을 변경.
- canvas.bind("<키이름>", lambda e: 이벤트): canvas에서 발생하는 키 입력을 bind한다.

각 위젯마다 속성이 다르고 이를 어떻게 활용하느냐에 따라 다른 형태를 갖기 때문에 이를 잘 암기하고 활용하는 것은 매우 중요하다.

추가적으로 outline, create_oval, create_rectangle, create_triangle는 각각 선, 타원, 직사각형, 삼각형을 화면 상에 그리는 함수로 테두리 색상(outline), 내부 색상(fill), 테두리 두께(width) 등의 속성을 갖는다.

※ 6. Numpy and Pandas

6.1 Numpy

- **스칼라**^{Scalar}: 크기만 가지고 방향 가지지 않는 수. 실수 정수 복소수 등이 예시. 한 개의 값
- **벡터**^{Vector}: 크기와 방향을 가지는 수. 여러 개의 값을 가지며 순서가 중요함.
- **행렬**^{Matrix}: 2차원 배열. 행과 열의 요소를 가짐. 각 요소는 스칼라이며, 행과 열에 따라 위치가 결정됨. 선형 변환, 선형 방정식, 데이터 표현 등에 널리 이용.

벡터나 행렬을 파이썬에서 다루기 위해서 주로 numpy를 많이 이용한다.

numpy는 파이썬에서 수치 계산을 위한 핵심 라이브러리로 널리 사용되는 도구로, 벡터와 행렬을 다루는데 굉장히 유용한 기능들이 다수 포함되어 있다. 즉, numpy를 사용하면 과학적이고 수치적인 계산을 효율적으로 수행할 수 있다.

우선, 기존에 벡터나 행렬을 표현하기 위해 사용했던 파이썬 리스트는 아래와 같은 특징이 있다.

- **높은 유연성**: list는 다양한 데이터 유형을 포함할 수 있으며 크기가 동적으로 조정될 수 있음.
- **여러 내장 함수**: 다양한 내장 함수와 method가 있어 데이터 조작과 분석이 쉬움
- **느린 속도**: 연산이 반복문 통해서 요소 별로 수행되기에 큰 데이터 셋에서 속도가 느림

- **많은 메모리 사용:** 각 요소에 대한 정보 저장하기 위해 추가적인 메모리 공간을 이용하여 큰 데이터 셋에서 메모리 사용이 비효율적임

파이썬 리스트는 굉장히 느린 편이다. 이는 리스트가 내부 구현상으로는 연속된 배열로 된 값이 아니라 연속된 주소의 배열을 갖고, 각 주소가 리스트의 각기 다른 요소를 가리키는 방식이기 때문이다.

반면, `numpy` 배열은 파이썬 리스트와 비교하여 아래와 같은 특징이 있다.

- **빠른 성능:** 배열이 C로 구현되어 있기 때문에 반복문 쓰지 않고 벡터화된 연산을 수행할 수 있어 처리 속도가 빠름.
- **적은 메모리 사용:** 연속된 공간에 데이터 저장하여 메모리 사용이 효율적
- **다차원 배열의 지원:** 다차원 배열 지원하여 행렬 연산 및 다차원 데이터 처리가 용이
- **다양한 수학 함수:** 다양한 수학 함수와 연산을 제공함
- **낮은 유연성:** 한 가지 데이터 유형만 포함할 수 있고, 크기를 동적으로 조정할 수 없음. 크기 변경하려면 새로운 배열 만들어야 함.
- **추가 기능의 부족:** 기본적인 수학과 배열 조작에 중점을 둔 함수밖에 없어서 추가 기능이 제한적일 수 있음.

`numpy` 배열은 파이썬 리스트에 비해서는 월등한 속도를 보여준다. 이때, 파이썬 리스트와 `numpy` 배열은 서로 장단점이 명확하게 구분되기 때문에, 시기적절하게 사용하는 것이 중요하다. 이때, `numpy`의 주요 함수는 다음과 같다.

함수 이름	설명
<code>array()</code>	배열 생성. 리스트나 튜플 객체 입력 받아 <code>numpy</code> 배열로 변환
<code>zeros()</code>	모든 요소가 0
<code>ones()</code>	모든 요소가 1
<code>arange()</code>	주어진 범위 내에서 일정한 간격으로 값이 증가하는 1차원 배열
<code>linspace()</code>	주어진 범위 내에서 지정된 개수에 맞춰서 일정한 간격으로 값 증가하는 1차원 배열
<code>reshape()</code>	배열의 차원과 크기 재조정
<code>transpose()</code>	전치

concatenate()	여러 배열을 연결함. 특정 축을 따라 배열을 연결할 수 있음.
sum()	모든 요소의 합
mean()	모든 요소의 평균
max, min()	최대, 최소
matmul()	행렬 곱
sort()	배열 정렬. 특정 축 기준으로 할 수 있음.
argmax()	최댓값의 index. 특정 축 기준으로 할 수 있음.
argmin()	최솟값의 index. 특정 축 기준으로 할 수 있음.

6.2 Pandas

pandas는 대규모 데이터를 조작하고 분석하기 위한 라이브러리이다. 강력한 데이터 구조와 데이터 조작 기능 제공하여 데이터를 쉽게 조작하고 분석할 수 있게 해준다. **시리즈**^{Series}는 1차원 배열과 같은 자료구조이다. 각 요소는 인덱스-값 쌍으로 구성되어 있으며 인덱스는 각 요소에 대해 고유한 숫자이다.

여러 개의 시리즈들을 모아서 **데이터프레임**^{DataFrame}을 구성할 수 있고, 이때 각 열은 서로 다른 데이터 유형을 가질 수 있다. 데이터프레임에서의 각 행은 하나의 데이터 쌍을 의미하고 행마다 인덱스를 가지고 있다.

함수 이름	설명
pd.DataFrame()	데이터프레임 생성 함수
head(n)	첫 n개 행을 반환 (기본값은 5)
tail(n)	첫 n개 열을 반환 (기본값은 5)
info()	기본 정보를 출력(열의 데이터 타입, 결측치 유무, 메모리 사용량)
describe()	숫자형 열에 대한 기술 통계 정보 출력(평균, 표준편차, 최대최소, 사분위수)
shape	행, 열의 크기 반환
columns	열의 index 반환
index	행의 index 반환
values	값을 2차원 배열 형태로 반환
loc[rl, cl]	label 이용하여 행과 열을 선택
iloc[ri, ci]	위치 이용하여 행과 열을 선택
isnull()	결측치 있는 위치를 확인하여 boolean 형태로 반환
dropna()	결측치 있는 행 또는 열을 삭제
fillna(v)	결측치를 value로 채움

`sort_values(c)` 열 기준으로 데이터프레임 정렬. 기본적으로 오름차순이며, `ascending=False`로 내림차순으로 정렬 가능.

References

- [1] Youn Eun Young. *CSED101 Lecture Presentation Slides*. POSTECH, 2024.
- [2] John V. Guttag. *Introduction to Computation and Programming Using Python, Third Edition*. MIT Press, 2021.